baton

FINAL REPORT · ROUND 3

Baton.

Every handover sentence carries its receipt.

What we planned, what we built, the three improvements we promised for Round 3 and exactly what was delivered, what changed and why, how we tested it, and what is still not verified.



PROBLEM	Prevent Critical Knowledge From Leaving With People
TEAM	Team Bramhastra · Deven Rikame (Team Leader) · Bhushan Shinde · Shawn Dmello
LIVE PRODUCT	baton-umber.vercel.app (demo workspace: **demo**)
REPOSITORY	github.com/deven065/Baton (public)
DATE	14 November 2026 · supersedes the Round 2 build report of 11 November 2026

CONTENTS

What this document covers

★ **How to test Baton (judges, 5 minutes)**

01 Summary: what changed since Round 1

02 The problem and who it affects

03 The solution and workflow, as built

04 Delivery against the Round 1 plan

05 Features added beyond the plan, and why

06 Round 3: the three improvements

07 Architecture and technology

08 Security, privacy and access control

09 Evaluation and testing

10 Deployment and how to try it

11 Plan status and what is not yet verified

12 Known limitations

13 Team and disclosures

14 Submission checklist

01 · SUMMARY

01 What changed since Round 1

In Round 1 we proposed an evidence-linked, permission-aware handover platform. In Round 2 it was built, tested and live. In Round 3 we delivered the three major improvements we named in the Round 2 form: **extraction that is measured and checked by a second model pass**, **meeting recordings as evidence**, and **one-click connections with automatic sync** that tells the lead when new evidence changes a handover. Section 06 says exactly what was delivered, what changed from our wording and why, and what we have not yet verified on the live site.

Live
Deployed web app, API and database, open to new organisations

130
Automated tests, run on every push in CI

2 sets
Of planted facts: 23 of 25 (easy) and 8 of 25 (hard, with traps) found offline; model results pending

0
Permission leaks and raw secrets stored, checked automatically

Area	Round 1 plan	Where we are now
Core workflow	Connect, reconstruct, verify, review, hand over	Built end to end, with an approval gate, SHA-256 versions, role-filtered views and acknowledgement
Sources	GitHub, task CSV, documents, Slack export; WhatsApp and Discord "planned"; live connectors "future"	All of these, plus **live Slack, Google Drive and Jira** connectors, **WhatsApp and Discord** exports, **meeting recordings** and automatic sync
Round 3 improvements	Named in the Round 2 form: measured model extraction, meeting recordings, one-click connections with automatic sync	**Built and tested** all three; live checks pending (section 06)
Data ownership	One shared database	Each organisation connects **its own PostgreSQL**; Baton keeps only the workspace name and an encrypted way in
AI	Groq, server-wide	Each organisation can **bring its own AI provider and key**; a fully offline mode needs no AI at all
Access control	One access policy on every read	Enforced by the API on every request, checked by an automated matrix of 70+ endpoints × 9 kinds of caller
Evaluation	Script against 25 planted facts	Two benchmark projects (the second harder, with traps and precision), a script that compares offline mode with models, and a test on every build

START HERE · FOR JUDGES · ABOUT 5 MINUTES

How to test Baton

Open **baton-umber.vercel.app**. On the sign-in page, click a demo person to sign in. There is nothing to type and nothing to install.

> The first visit can take up to a minute while the free server wakes up. To switch person, click the round avatar at the top right, choose **Sign out**, then click the next person.

#	Sign in as	Do this	What it proves
1	**Neha Kulkarni** project lead	Open **Review** and click any claim: its exact source quote is shown. Open **Conflicts**: two sources disagree on the launch date. Open **Knowledge risk**.	Every sentence has a receipt; contradictions are caught, not hidden.
2	**Priya Sharma** person leaving	Answer one exit-interview question.	She is asked only what the sources could not answer; her answer becomes evidence.
3	**Arjun Mehta** incoming engineer	Read **Start here**, tick an item in **Checklist**, then in **Ask** type *When is the launch?*	The successor gets a plan and answers, each with its source.
4	**Rahul Desai** incoming designer	Open the same handover. Note the line saying some items are above his clearance.	Each person sees only what they are allowed to; the server enforces it.
5	**Org Admin**	Open **People & groups**, then **Audit log** and filter by event, person or date.	Who did what, and when, is recorded in the organisation's own database.

> **New in Round 3.** Connections (Organisation, Connections) has a **Jira** card and, once the operator has registered the apps, one-click buttons. On Sources, **Meeting recording** accepts audio or a transcript file; the demo files are in `demo/recordings/` in the repository. Live sources show "Updates itself every hour" with a switch. Try these in your own workspace; the shared demo is read-mostly.

Optional: try it as your own organisation (about 3 minutes)

1. Create a free project at **neon.tech**. In **Connect**, turn **Connection pooling off** and copy the connection string.
2. In Baton, click **Create an account**, choose a workspace name, paste the connection string, and create your admin account.
3. Add a person, a project and a source, then reconstruct a handover. Your data stays in your own database.

> The demo is shared by every visitor, so changing passwords, people or settings is switched off there. Everything else works. Use your own workspace to try those.

02 · CONTEXT

02 The problem and who it affects

When someone leaves a project, the documents stay. What leaves is the reasoning, the unfinished work, the contacts and the access, because those live in scattered tools or in one person's head. Our understanding of the problem is unchanged from Round 1; building Baton confirmed it.

What is lost	Where it hides today	Why documents miss it
Why decisions were made	Meetings, pull-request reviews, chat	Docs record the outcome, not the reason or the options rejected
Unfinished work	Open pull requests, branches, half-moved task cards	The status is visible; the next step exists only in one head
Dependencies and contacts	Direct messages, personal email, vendor calls	Private by nature, so never written into shared docs
Recurring problems	Scattered across months of chat	No single message looks important; only the pattern does
Access and ownership	Accounts on personal emails, admin roles	Nobody writes "this account is under my Gmail" until it is too late

Who uses Baton

User	Role in Baton	What Baton gives them
Person leaving e.g. a graduating tech lead	Answers gap questions	6–12 targeted questions instead of a 20-page document in their final week
Project lead	Reviewer and approver	A handover they can trust and sign off, with conflicts surfaced and nothing leaked
Successor engineer, designer, volunteer	Recipient	Start here, a Day 1 / Week 1 / Month 1 checklist, who to ask, and answers with sources
Organisation admin	Runs the workspace	People, groups, clearances, projects, connections, AI, billing and the audit log

All three departure scenarios from Round 1 are supported: **planned** (full flow with exit interview), **sudden** (works from sources alone; questions go to the lead) and **split** (one approved handover, a different filtered view for each successor). Primary setting: student tech clubs and volunteer teams, where turnover is guaranteed. The same product serves startups and NGOs, which is why we added self-service organisations and plans (section 05).

03 The solution and workflow, as built

Baton reads a project's authorised sources, rebuilds the project state as typed claims, and attaches a verbatim source quote to every claim. Code, not the AI, checks each quote. Gaps become a short exit interview, a lead approves, and each successor sees only what they are allowed to.

> **Our principle, unchanged:** the AI proposes, code verifies, humans approve. Anything unproven is labelled as such.

1 Connect
Authorised sources, each with a sensitivity and access groups; redacted on the way in

2 Reconstruct
Task-board and repository signals, then AI extraction into 11 claim types

3 Verify
Quotes located by code and checked by a second model; duplicates merged; conflicts linked; gaps become questions

4 Review
Leaver answers; lead accepts, edits or rejects; approval gate; frozen hashed version

5 Hand over
Role-filtered view, checklist, Ask, acknowledgement, progress

The reconstruction pipeline

Stage	What happens
Signal	Facts taken straight from systems of record (task board, GitHub), with no AI
Extract	Redacted passages, wrapped as untrusted data, are sent to the model in batches grouped by access class
Verify	Code locates every quoted sentence in its passage; unmatched quotes are dropped
Consolidate	Duplicate claims are merged
Check Round 3	A second model pass asks whether each verified quote really supports its claim. It can only make a claim weaker: "not supported" turns a fact into an inferred claim that a person must decide. Skipped in offline mode
Conflict	Claims that contradict each other are linked, and block approval until resolved
Score	Status and confidence computed by code: source trust, independent sources, quote match, recency, conflicts
Gap	6–12 exit-interview questions for what the sources cannot answer
Timeline	Dated decisions and changes put in order

Nine stages: Round 1 described nine, Round 2 merged two into eight, and Round 3 added the model check. Stages are idempotent and resume after a crash.

The evidence model

Status	Rule (computed by code)	Shown to the successor as
Verified	Two or more independent sources, or one system of record; all quotes match	Fact
Single source	One matching quote from notes or chat	Fact, marked "one source"
Inferred	The model's reasoning with no direct quote	"Likely, not confirmed"; needs a decision
Conflicting	Sources disagree and it is unresolved	Blocks approval until resolved
Missing evidence	An expected fact has no support	"Unknown"; becomes an exit question
Stale	Superseded by newer evidence	Hidden by default, kept in history

04 Delivery against the Round 1 plan

Every feature we listed in the Round 1 proposal is built. The table follows the proposal's own feature list and its brief capabilities.

Feature (Round 1)	What is built	Status
Source connectors	GitHub (read-only API), task CSV (Jira, Trello, GitHub Projects), documents (Markdown, text, PDF, Word), meeting notes, Slack export; live progress while ingesting	Done
WhatsApp and Discord exports	WhatsApp "Export chat" (.txt or .zip; Android and iPhone formats) and Discord (DiscordChatExporter JSON or CSV), recognised automatically	Done was "planned"
Redaction	Emails, phone numbers, API keys, passwords and private keys replaced with placeholders before storage and before any AI call; originals kept encrypted	Done
Claim extraction	All 11 claim types: decision, open work, owner, risk, dependency, blocker, recurring issue, contact, open question, process, access to transfer	Done
Citation verifier and confidence	Every quote located in its passage by code; status and a transparent confidence score	Done
Conflict and gap detection	Contradictions linked and shown side by side; missing owners and evidence become questions; claim merging	Done
Exit interview	Targeted questions; answers redacted, refused if they contain keys or passwords, stored as testimony and re-scored	Done
Decision timeline	Dated decisions, changes and unresolved discussions	Done
Review workspace	Accept, edit, reject, needs-info; highlighted evidence; keyboard shortcuts; full revision history; protection against two reviewers overwriting each other	Done
"View as" preview	The lead sees exactly what each recipient will see	Done
Approval and versions	Approval gate with blockers and warnings; frozen versions with a SHA-256 hash; diff between versions	Done
Recipient handover	Start here, Day 1 / Week 1 / Month 1 checklist linked to claims, ownership map, acknowledgement; shares that expire or are revoked	Done
Audit log	Every change and every view of a share recorded in the organisation's own database	Done
Reliability	AI retry with back-off, response cache, offline mode, one repair attempt for invalid output, database job queue that resumes after a crash	Done
Local embeddings	all-MiniLM-L6-v2 in the JVM; text never leaves the server for embedding	Done off on the free host
Permission-leak test	Automated test of every persona against every endpoint	Done
Evaluation script	25 planted facts, a scoring script and a test that runs on every build (section 09)	Done
Docker, CI and hosting	Docker Compose for local setup; GitHub Actions for tests and deployment; Vercel, Render and Neon	Done

The Round 1 plan allowed us to cut the prompt-injection screen, the diff view and claim merging if time ran short. We cut none of them.

05 Features added beyond the plan, and why

While building, we tested Baton against a simple question: would a real club, startup or NGO trust it with their chats, contracts and accounts? The answer shaped the features below. None were in the Round 1 scope; each solves a concrete adoption or trust problem.

1 · Each organisation keeps its data in its own database New

WHAT Self-service sign-up: an organisation names its workspace and connects an empty PostgreSQL database in its own account (Neon, Supabase, AWS RDS, Cloud SQL, Azure or self-hosted). Baton creates the tables there. Everything (people, sources, claims, versions, the audit log) is stored in that database. Baton keeps only the workspace name and the connection details, encrypted. Each workspace has its own AES-256-GCM key. An admin can disconnect at any time and keep everything.

WHY Handover data is the most sensitive data a team has: private chats, vendor contracts, account ownership. Organisations will not hand that to a third-party database. Keeping it in their own database removes the biggest reason to say no, and makes leaving Baton painless. Before connecting, Baton checks the database (version, empty, pgvector, create permission) and explains the exact fix if something is missing.

2 · Organisation administration New

WHAT People with clearances and groups, projects with leads and teams, one-time temporary passwords that must be changed on first sign-in, password reset, deactivation, and an audit log readable by admins. Guard rails: a workspace always keeps an active admin; a project always keeps a lead.

WHY The Round 1 demo used fixed personas. Real permission-aware filtering only means something when an organisation can manage who has which clearance and group, so we built that management.

3 · Bring your own AI New

WHAT An admin connects the organisation's own provider and key: Groq, OpenAI, OpenRouter, or any OpenAI-compatible API, including self-hosted models. Keys are encrypted; usage is shown for 30 days. Without any AI, a deterministic offline mode still runs the whole workflow.

WHY Confidential teams need to choose where their text goes, and who pays for it. A single server-wide AI key would make Baton's costs grow with every customer and force everyone onto one provider. "Self-hosted models" was in our future scope; this delivers it now.

4 · Live connectors: Slack and Google Drive New

WHAT Besides the GitHub API, Baton reads a Slack channel (last 90 days with threads) and a Google Drive folder (Docs and text files) using the organisation's own read-only credentials, and can sync them again. Baton never writes to these tools.

WHY Exports go stale the day they are made. Live sources are what make "living handovers" (section 06) possible: re-sync, detect what changed, propose a new version. Round 1 listed live connectors as future scope.

5 · WhatsApp and Discord chat exports Brought forward

WHAT The chat-export source recognises WhatsApp (Android and iPhone, 12- or 24-hour clocks, either date order, with or without media) and Discord exports automatically, and turns them into the same quoted, dated messages as Slack.

WHY Indian student clubs and NGOs, our primary users, coordinate on WhatsApp and Discord far more than on Slack. Without these, the knowledge that matters most to them stays out of reach.

6 · Successor progress New

WHAT Each recipient's checklist progress by phase, with due dates (Day 1, Week 1, Month 1 after sharing), overdue items and questions still waiting for an answer.

WHY Acknowledgement tells a lead the handover arrived, not that it worked. Progress shows whether the successor is actually taking over, while the person leaving can still help.

7 · Signed export and verification New

WHAT Any approved version can be downloaded as a zip: the snapshot exactly as hashed, the checklist, shares, the audit trail, a printable report and a signed manifest. "Verify an export" checks every file and the signature and catches any change.

WHY A handover is a record of responsibility. When an account is lost or a decision is disputed months later, an organisation needs proof of what was handed over, and that it has not been altered.

8 · Access control that cannot be bypassed Hardened

WHAT Every endpoint checks the caller on the server. Role, clearance, groups and active status are read from the database on every request, so removing someone or lowering their clearance applies immediately, even to people already signed in. Hidden items return "not found". A temporary password opens nothing but the password change. An automated test calls more than 70 endpoints as nine kinds of caller (admin, lead, leaver, member, recipient, outsider, another organisation's admin, a forged token and no token) and fails on any request that should be refused but is not.

WHY "Permissions that cannot leak" was one of our three core ideas. Hiding a button is not security; typing a URL or replaying an old token must not get around it either.

9 · Everyday essentials New

WHAT Password reset and invitations by email, a notifications bell (exit answers, recipients opening or acknowledging), header search for claims and people, an account menu on the avatar (account settings, organisation settings, sign out), a handover switcher and rename for leads, an audit log that filters by event, person, date and text and downloads as CSV, "keep me signed in", and rate limits on sign-in and other sensitive actions.

WHY These are what users expect from any product they sign in to. Without them, a real team cannot run Baton without our help.

10 · Plans and billing New

WHAT A Free plan (5 people, 1 project, 10 sources) and a Team plan (100 people, 50 projects, 1,000 sources), paid through Razorpay payment links. Baton never sees card details; payment notifications are verified by signature. Limits only stop adding; nothing is hidden when a plan lapses.

WHY A product that clubs and NGOs depend on must be able to sustain itself after the event. The Free plan covers a typical club, so the people we built Baton for never have to pay.

11 · A guided, considered experience New

WHAT A "Your next step" card on the dashboard, and "Next: ..." in the sidebar on every page, name the one thing to do now (add your team, add sources, start a handover, resolve conflicts, approve, share), say why in one sentence and take you there; the checklist is ticked from real data. Plus a distinct visual identity (a lowercase "b" handing on its baton), consistent components, smooth transitions and loading states, a guided sign-up, and a public guide for connecting a database.

WHY Our success measure is how fast a newcomer can name their first task and who to ask. A tool a club depends on must be usable without a manual; a clear, calm interface is part of that, not decoration.

06 Round 3: the three improvements

In the Round 2 form we named three major improvements. The Round 1 commitments (Ask the handover, living handovers, the knowledge-risk radar) were already delivered in Round 2. Below, for each improvement: what we promised, what is built, what changed from our wording and why, and how it was verified.

Improvement 1 · Extraction you can measure

Built and tested

PROMISED Run reconstruction with a real language model; add model-graded quality checks on top of the code-verified quotes; publish recall, precision and cost for offline mode against at least two models on our 25-fact benchmark, expanded with a second, harder synthetic project.

BUILT (a) A new **Check** stage: after code proves a quote exists, a second model pass judges whether the quote supports the claim as written (supported, partly, not supported). A "not supported" verdict stops the claim being shown as fact until a person decides, and the Review page shows the verdict and the model's reason. It can only weaken a claim, never strengthen one. (b) **Project Godavari**, a harder benchmark: a hackathon-platform handover with 25 plainly worded facts, some in Hinglish, **seven traps** (a joke, a hypothetical, a rumour, a rejected idea, sarcasm, and two values that were later changed), an injected instruction, and an answer key that lists every other true statement, so **precision** can be measured. (c) `demo/benchmark.py` builds each project fresh and reconstructs it once per configuration, reporting recall, precision, traps, trust checks, model-check verdicts, tokens, time and list-price cost. (d) Token use is recorded per run and shown on the Reconstruct page.

MEASURED SO FAR (OFFLINE RULES) Annapurna: 23 of 25 facts. Godavari: **8 of 25** facts, 11 of 11 claims shown as fact correct, 0 of 7 traps shown as fact, all trust checks passing. The benchmark also found a real bug (a token pasted in chat as "token if anyone needs it: ..." was not redacted), now fixed and tested.

NOT YET MEASURED **Recall, precision and cost with real models.** The harness is ready (`python demo/benchmark.py --configs offline,groq:llama-3.3-70b-versatile,groq:openai/gpt-oss-120b`) but it needs a Groq API key, which only the team can create. The model rows will be added to `demo/benchmark-report.md` in the repository, and the production server still runs in offline mode until a key is set. We do not claim model numbers we have not measured.

WHAT CHANGED "Run with a real model in production" depends on a key, not on code, so we delivered the capability and the measurement tooling, and report the numbers only once they exist. The harder project showed the gap clearly: offline rules find 32% of the hard facts, which is the case for a model.

Improvement 2 · Meeting recordings as evidence

Built and tested

PROMISED Upload a meeting recording; Baton transcribes it (Whisper through the organisation's AI provider), labels speakers, redacts it like any other source, and extracts claims that cite the exact timestamp; clicking a claim plays that moment.

BUILT A new source type, **Meeting recording**. Audio (mp3, m4a, wav, webm, ogg, flac) is transcribed with Whisper through the organisation's own provider (Groq or OpenAI), or the server's key. A transcript exported by Zoom, Teams or Meet (.vtt, .srt) works with **no AI at all** and keeps its speaker names. The text is stored as `[hh:mm:ss] Speaker: text` lines and redacted like any source. Every quote from a recording carries the second it was said, and the Review and Conflicts pages show a **play button (for example 0:50)** that plays the recording from that moment. The audio is kept encrypted with the workspace key; every play is audited; the lead can **delete the audio and keep the transcript**. A 72-second demo call with two voices and a matching transcript are in `demo/recordings/` .

WHAT CHANGED Whisper does not tell voices apart, so "labels speakers" became **suggested** speaker names: the model names a person only when the conversation shows who it is (an introduction, being addressed by name), otherwise it writes "Speaker A", and the transcript says the names are suggestions to check. We also added transcript files, because many teams already have them and they need no AI.

VERIFIED Automated tests with a fake Whisper and a fake chat model: transcription, speaker names, timestamps on evidence (a quote said at 0:50 reports second 50), playback returns the exact uploaded bytes, only the lead can delete audio, audio is refused with a clear message when no provider is connected, and transcript files work without AI. We also played a claim's moment in the browser against a local stand-in for Whisper. **Not yet verified:** a real Whisper call; that needs the Groq key. The audio goes to the AI provider before redaction (speech cannot be redacted before it is transcribed), so organisations should upload only recordings they may share with that provider; the transcript is redacted.

Improvement 3 · One-click connections and automatic sync Built and tested

PROMISED "Sign in with Slack / Google / GitHub" (OAuth) instead of pasted keys, a new Jira connector, and scheduled incremental sync that feeds the existing drift detection: when new evidence changes an approved handover, the lead is notified and offered the next version.

BUILT **One click:** on Organisation, Connections an admin clicks "Add to Slack", "Connect with GitHub" or "Connect with Google", approves read-only access on that tool's own page and returns to Baton with no key to copy. The sign-in carries a signed ten-minute state that is deliberately not a sign-in token; the token is stored encrypted exactly like a pasted one; Google keeps a refresh token and renews access when syncing. **Jira:** a live connector for a Jira Cloud project; open issues owned by the person leaving, or by nobody, become handover items like a task-board export. **Automatic sync:** live sources (GitHub, Slack, Google Drive, Jira) are read again every hour by default, with a switch per source. Each read is fingerprinted, so nothing is re-indexed when nothing changed. **When something changed:** the lead gets a notice (dashboard, approval page, bell and email when on); a handover still in review is reconstructed again by itself, keeping the claims already reviewed; an approved or shared handover is left alone, its recipients keep the frozen version, and the lead is offered **Start version N+1**. **When a read fails** (a revoked token, an outage) the source keeps its last good copy and the lead gets one notice, cleared when a read works again.

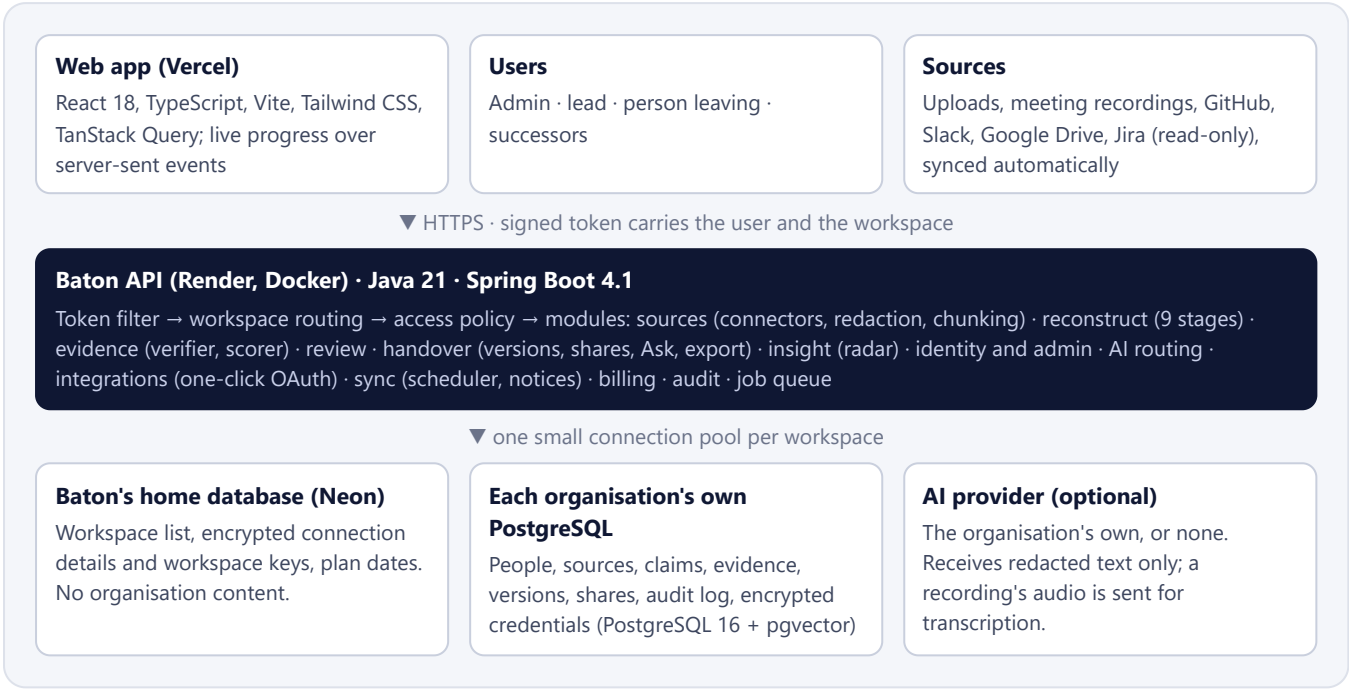
WHAT CHANGED Jira connects with an API token (site, account email, token), not OAuth, because Atlassian's OAuth needs a separate app and token rotation that we judged less useful than finishing the sync. One-click connections need the **server's operator to register one app per tool** (Slack, GitHub, Google) and set its client id and secret; until then the page shows the paste-a-key form, so nothing is broken without them. For GitHub we recommend a read-only GitHub App rather than an OAuth App, which can write.

VERIFIED Automated tests with fake Slack, GitHub and Google servers: the connection is stored, the stored token works, and a forged, tampered, expired or wrong-tool state is refused and stores nothing. A fake Jira covers the first read, an unchanged check that re-indexes nothing, new evidence (notice, the handover in review reconstructed again with the new issue as a claim, an approved handover left alone), a failing read, and the lead-only controls. We watched the whole loop in the browser: a new Jira issue appeared and about a minute later the dashboard showed "new evidence" with a button to start the next version. **Not yet verified:** a real one-click sign-in (the OAuth apps are not registered yet) and a real Jira site. The schedule runs inside the API process, so on the free host that sleeps when idle it only runs while the API is awake; a free uptime monitor keeps it running.

Earlier commitments, still delivered: Ask the handover (cited answers from the approved version only), living handovers (the lead is warned when the approved version is out of date and moves every recipient in one click) and the knowledge-risk radar (bus factor, single-source topics, access only one person mentions, topics gone quiet).

07 Architecture and technology

One React web app talks to one Spring Boot application (a modular monolith). Every request passes the token filter, is routed to the organisation's own database, and is checked by the access policy.



Layer	Choice	Change from Round 1, and why
Backend	Java 21, Spring Boot 4.1, Spring Security, JPA, Flyway	Spring Boot 4 instead of 3: the current major version when the build began
Database	PostgreSQL 16 with pgvector and full-text search	One database per organisation instead of one shared database (section 05)
AI	Any OpenAI-compatible provider, per organisation; Whisper for recordings; offline mode	Not tied to Groq; each organisation chooses. Round 3 added a second model pass and speech to text
Frontend	React 18, TypeScript, Vite, Tailwind CSS 4, own component set	Own components instead of shadcn/ui, for a consistent identity
Parsing, embeddings, delivery	Apache Tika, Commons CSV; all-MiniLM-L6-v2 in the JVM; Docker, GitHub Actions; Vercel, Render, Neon	As planned, on free tiers

08 Security, privacy and access control

The trust requirements from the brief are enforced by code, not by the AI and not only in the interface.

Requirement	How Baton meets it
Don't present assumptions as truth	A claim is shown as fact only if its quote is found word for word in a source the reader may see. Inferred claims are labelled and need a reviewer's decision before approval.
Don't expose confidential information	A claim inherits the highest sensitivity of its evidence and is hidden if any passage it cites is hidden. Search filters before ranking. Permissions are checked on the server on every request.
Keep secrets away from AI and storage	Emails, phone numbers, keys, passwords and private keys are redacted before anything is stored or sent to a model; originals are encrypted with the workspace key. Exit answers containing keys or passwords are refused.
Resist manipulation	Source text is sent as untrusted data; passages that try to instruct the model are flagged and never change a status.
Let authorised people correct and approve	Per-claim accept, edit, reject or needs-info with full history; approval gate; frozen hashed versions; diff.
Traceability and audit	Every claim links to its source; every change and every view of a share is logged in the organisation's own database; adding a source requires confirming authorisation to share it.
Data ownership	Organisation data lives in its own database with its own encryption key. Disconnecting removes Baton's access and keeps the data.

Additional safeguards

- **Accounts:** BCrypt password hashes; minimum 12 characters; one-time reset links stored only as hashes; rate limits on sign-in, sign-up and other sensitive actions; unknown workspaces answer exactly like a wrong password.
- **Network safety:** organisation databases and AI providers must be public hosts (no private or loopback addresses); AI providers must use HTTPS.
- **Logs:** errors are logged by type and code location only, never with organisation content.
- **Payments:** payment notifications are trusted only with a valid signature, the right amount and currency, and are counted once.
- **Shared demo:** the demo workspace's passwords are public, so changes that would break it for the next visitor (passwords, deactivating people, removing team members, revoking shares, AI, connections, plan) are refused there. The handover workflow stays fully usable.
- **Deployment secrets:** the server refuses to start against a remote database with the development token secret, and reports the development master key, so a misconfigured deployment cannot run unnoticed.

09 Evaluation and testing

We measure Baton against the targets we set in Round 1, on two synthetic projects with facts we planted on purpose: "Project Annapurna" (a college club's canteen pre-order app) and, new in Round 3, the harder "Project Godavari" (a hackathon platform).

The planted-facts evaluation

Seven synthetic sources (meeting notes, a Slack export, a WhatsApp group, a Discord channel, a task board, architecture notes and a restricted vendor contract) contain **25 planted facts**, one of every claim type, a launch-date conflict, a task with no owner, an injected instruction and a fake API key. A fact counts as found only when a claim of the right type contains its key words.

Measure (Round 1 target)	Target	Result
Planted facts found (recall)	Report in Round 2	**23 of 25 (92%)** in offline mode, with no AI model
Claims shown as fact with a matching quote	100%, by design	**100%** (20 of 20 in the evaluation run)
Permission leaks	0	**0**: restricted vendor facts never reach the person leaving; the access matrix passes
Raw secrets stored or sent to AI	0	**0 of 5** planted secrets found in any stored passage
Injected instructions obeyed	0	**0**: flagged as a warning, never turned into a claim
Time for a new user to name their first task	Measure with five users	Planned for usability testing (section 11)

> **The two facts offline mode misses** on Annapurna are plain statements with no signal word ("secrets live in the Render dashboard", "renewal is due on 2026-12-01"). We left the data unchanged rather than tune it to our rules; a model is expected to find both.

The harder benchmark: Project Godavari (Round 3)

Seven sources with **25 plainly stated facts** (some in Hinglish) and **seven traps**. The answer key lists every other true statement too, so we can measure precision: of the claims shown as fact, how many are correct.

Measure	Offline rules	With language models
Planted facts found (recall)	**8 of 25 (32%)**	Pending: needs the team's Groq key
Claims shown as fact that are correct (precision)	**11 of 11 (100%)**	Pending
Traps shown as fact	**0 of 7**	Pending
Trust checks (secrets, injection, restricted facts)	**5 of 5** pass	Pending
Tokens and list-price cost	0 (no model)	Pending

Offline rules are precise but find little; that gap is what a model is for. `demo/benchmark.py` fills the right-hand column; we publish no number it has not produced. A trap the offline rules skip counts as avoided, so the model results are where traps become a real test.

Automated tests and continuous integration

Area	What is tested
Evidence and ingestion	Quote matching, scoring rules, redaction, chunking, every connector format (Slack, WhatsApp, Discord, CSV, documents)
Access	Every endpoint × nine kinds of caller; temporary passwords; permission changes applying to existing sessions
Workflow	Reconstruction on the demo data, review, approval blocking, versions, shares, Ask, signed-export tampering
Organisations	Workspace isolation, database checks, disconnect, password links, plan limits, payment signatures
Evaluation	The 25 planted facts and the four trust checks, on every build; the model check with a scripted judge (disputed claims stop being shown as fact and block approval)
Recordings	Transcript parsing (Teams and Zoom VTT, SRT), fake Whisper and speaker labelling, timestamps on evidence, playback byte for byte, lead-only deletion
Connections and sync	One-click sign-in against fake Slack, GitHub and Google (forged, expired and wrong-tool state refused); fake Jira for first read, unchanged check, new evidence, approved handover left alone, failing read, switch off

130 automated tests run against a real PostgreSQL with pgvector (started by Testcontainers). GitHub Actions runs them, plus the frontend type-check and build, on every push and pull request, and publishes the evaluation report with each run.

10 · TRY IT

10 Deployment and how to try it

Part	Where
Web app	https://baton-umber.vercel.app (Vercel, deployed from GitHub on every push)
API	Render, Docker image built from the repository; deployed by GitHub Actions on every push
Baton's home database	Neon PostgreSQL (workspace list only)
Source code	https://github.com/deven065/Baton, with full documentation in `docs/DOCUMENTATION.md`

Three ways to try Baton

1 · Demo workspace

On the sign-in page, pick a demo person in workspace **demo**: the project lead, the person leaving, two successors with different clearances, or the admin. Compare what each one sees.

2 · Your own organisation

"Create an account": name the workspace, connect an empty PostgreSQL database (a free Neon project works; the in-app guide shows how), and create the first admin.

3 · Run it locally

`docker compose up -d`, start the API and the web app, then `bash demo/load-demo.sh` and `python demo/evaluate.py` to reproduce the evaluation.

The API runs on a free plan that sleeps when idle; the first request after a pause can take up to a minute while it wakes, and the web app shows this.

11 Plan status and next steps

Round 2 milestone	Done means (Round 1)	Status
M1 Skeleton	Docker Compose, schema, login, demo personas, React shell, first deployment	Done
M2 Connect	Connectors, chunking, redaction, search indexing, background jobs with live progress	Done
M3 Reconstruct	Repository signals, AI extraction, citation verifier, scoring, gap questions, timeline	Done
M4 Review	Review workspace, evidence drawer, edits with history, exit interview	Done
M5 Handover	Approval gate, frozen versions, recipient views, checklist, acknowledgement	Done
M6 Harden	Permission-leak test, offline AI mode, evaluation report, deployment	Done

Round 3 status

Improvement	State	Left to verify
1 · Measured extraction	Check stage, harder benchmark, benchmark script and cost tracking built and tested; offline results measured	Model results (Groq key), model on the live server
2 · Meeting recordings	Audio and transcript upload, timestamps, click to play, audit, delete audio built and tested	A real Whisper transcription (Groq key)
3 · One-click connections, Jira, automatic sync	OAuth flow, Jira, scheduled incremental sync, notices and next-version offer built and tested	Real OAuth sign-ins (apps to register), a real Jira site

Remaining work

Item	Plan
User research	Survey of students and club leads (target 40+ responses) and five interviews
Usability testing	Five users given only the link; time to name their first task and who to ask
Recall with a language model	Run `demo/benchmark.py` with a connected AI provider on both projects and publish the numbers next to the offline ones
Contribution logs	Each member's log added to the repository
Accessibility pass	Keyboard-only review, contrast, status shown in text and not colour alone
Submission material	Three-minute demo video, privacy and responsible-design write-up, final presentation

Future scope after the event: Notion and live Discord connectors; real speaker separation for recordings; Hindi and Marathi; weekly "what changed" digests; offboarding for HR teams.

12 · HONESTY

12 Known limitations

- Knowledge that was never written down and that the person leaving does not recall cannot be recovered; Baton labels it unknown rather than guessing.
- Extraction is English-first. Quote-matching thresholds are tuned on synthetic data and will need tuning on real data.
- Offline mode is rule-based: it found 23 of 25 planted facts on the easy project and 8 of 25 on the harder one, and misses statements without a signal word. The production server runs in offline mode until an AI key is set. Model-graded checks and model recall figures need a provider.
- Embeddings are off on the free hosting plan (512 MB memory), so production search is keyword-only.
- Rate limits and per-workspace AI clients live in memory; several API instances would need a shared store.
- One-click connections need the operator to register an app with each tool; until that is done on the live server the buttons are hidden and pasting a key still works. Jira connects with an API token only.
- Automatic sync runs inside the API process, so on a free host that sleeps when idle it only runs while the API is awake. A recording's audio is sent to the AI provider before redaction; speaker names are AI suggestions.
- Slack reads the last 90 days of one channel per source; Drive reads Docs and text files, not PDFs or Sheets; Jira reads the 500 most recently updated issues of one project.
- Payments are one month at a time, without automatic renewal or invoices.
- The knowledge-risk radar depends on sources naming their authors.

13 · TEAM

13 Team and disclosures

Member	Role	Responsibilities
Deven Rikame Team Leader	Backend developer	Planning and coordination; backend APIs; authentication and access policy; review, approval and versioning; sharing and recipient views; submissions and presentation
Bhushan Shinde	Database and AI prompts	PostgreSQL schema and migrations, indexes, full-text and vector search; prompt design for extraction, conflicts, gap questions and the continuation plan; the evaluation against planted facts
Shawn Dmello	Frontend developer, deployment	React screens from source connection to the recipient handover; Docker, GitHub Actions and cloud deployment
All three	Shared	User research, the synthetic demo dataset, usability testing, documentation and the demo video

Disclosures

Topic	Statement
AI assistance	Claude (Anthropic) was used throughout the build as an AI assistant: for guidance and planning, and to generate parts of the code from our own specification. The team set the design and requirements, reviewed and tested the code, and is responsible for it. Inside the product, AI models run only when an organisation connects its own provider, and only on redacted text.
Data	All demonstrations use synthetic data ("Project Annapurna", "Project Godavari"). The demo call uses computer-generated voices. No real club or company messages or recordings are used.
Libraries and services	Spring Boot, React, PostgreSQL with pgvector, LangChain4j (embeddings), Apache Tika, Commons CSV, Tailwind CSS, TanStack Query, Testcontainers, Docker, GitHub Actions, Vercel, Render, Neon, Razorpay and Resend, with Whisper, Slack, GitHub, Google and Jira APIs used when an organisation connects them, under their licences and terms.
Repository	Developed in a public repository with frequent commits: github.com/deven065/Baton

> **In one line:** Baton does what we promised in Round 1 and the three improvements we promised for Round 3: extraction that is measured and checked, meeting recordings that play the moment a claim was said, and connections that keep a handover current on their own. What we have not verified yet, we say so.

14 · Submission checklist

Item	Where
Working project	baton-umber.vercel.app/login (demo workspace: **demo**)
Source code	github.com/deven065/Baton (public)
Updated PDF	This document
Completed improvements	Section 06: improvements 1, 2 and 3, each with what was promised, built, changed and verified
Technical details	Sections 03 and 07; full reference in `docs/DOCUMENTATION.md`
Testing notes	Section 09 (130 automated tests, two benchmark projects) and section 11 (what is not yet verified)
Team contributions	Section 13. The event asks each member to file their own entry with their role and the part they built; the repository history is public.